waving your hands in the air and trying to sound authoritative. The advice is weak because it is hard to precisely explain the glue that holds the classes and modules together into a single component. Weak though the advice may be, the principle itself is important, because violations are easy to detect—they don't "make sense." If you violate the REP, your users will know, and they won't be impressed with your architectural skills.

The weakness of this principle is more than compensated for by the strength of the next two principles. Indeed, the CCP and the CRP strongly define the this principle, but in a negative sense.

THE COMMON CLOSURE PRINCIPLE

> *Gather into components those classes that change for the same reasons and at the same times. Separate into different components those classes that change at different times and for different reasons*.

This is the Single Responsibility Principle restated for components. Just as the SRP says that a *class* should not contain multiples reasons to change, so the Common Closure Principle (CCP) says that a *component* should not have multiple reasons to change.

For most applications, maintainability is more important than reusability. If the code in an application must change, you would rather that all of the changes occur in one component, rather than being distributed across many components.[1] If changes are confined to a single component, then we need to redeploy only the one changed component. Other components that don't depend on the changed component do not need to be revalidated or redeployed.

The CCP prompts us to gather together in one place all the classes that are likely to change for the same reasons. If two classes are so tightly bound, either physically or conceptually, that they always change together, then they belong in the same component. This minimizes the workload related to releasing, revalidating, and redeploying the software.

This principle is closely associated with the Open Closed Principle (OCP). Indeed, it is "closure" in the OCP sense of the word that the CCP addresses. The OCP states that classes should be closed for modification but open for extension. Because 100% closure is not attainable, closure must be strategic. We design our classes such that they are closed to the most common kinds of changes that we expect or have

experienced.

The CCP amplifies this lesson by gathering together into the same component those classes that are closed to the same types of changes. Thus, when a change in requirements comes along, that change has a good chance of being restricted to a minimal number of components.

SIMILARITY WITH SRP

As stated earlier, the CCP is the component form of the SRP. The SRP tells us to separate methods into different classes, if they change for different reasons. The CCP tells us to separate classes into different components, if they change for different reasons. Both principles can be summarized by the following sound bite:

> *Gather together those things that change at the same times and for the same reasons. Separate those things that change at different times or for different reasons.*

THE COMMON REUSE PRINCIPLE

> *Don't force users of a component to depend on things they don't need.*

The Common Reuse Principle (CRP) is yet another principle that helps us to decide which classes and modules should be placed into a component. It states that classes and modules that tend to be reused together belong in the same component.

Classes are seldom reused in isolation. More typically, reusable classes collaborate with other classes that are part of the reusable abstraction. The CRP states that these classes belong together in the same component. In such a component we would expect to see classes that have lots of dependencies on each other.

A simple example might be a container class and its associated iterators. These classes are reused together because they are tightly coupled to each other. Thus they ought to be in the same component.

But the CRP tells us more than just which classes to put together into a component: It also tells us which classes *not* to keep together in a component. When one component uses another, a dependency is created between the components. Perhaps the *using* component uses only one class within the *used* component—but that still doesn't weaken the dependency. The *using* component still depends on the *used* component.

Because of that dependency, every time the *used* component is changed, the *using* component will likely need corresponding changes. Even if no changes are necessary to the *using* component, it will likely still need to be recompiled, revalidated, and redeployed. This is true even if the *using* component doesn't care about the change made in the *used* component.

Thus when we depend on a component, we want to make sure we depend on every class in that component. Put another way, we want to make sure that the classes that we put into a component are inseparable—that it is impossible to depend on some and not on the others. Otherwise, we will be redeploying more components than is necessary, and wasting significant effort.

Therefore the CRP tells us more about which classes *shouldn't* be together than about which classes *should* be together. The CRP says that classes that are not tightly bound to each other should not be in the same component.

RELATION TO ISP

The CRP is the generic version of the ISP. The ISP advises us not to depend on classes that have methods we don't use. The CRP advises us not to depend on components that have classes we don't use.

All of this advice can be reduced to a single sound bite:

> *Don't depend on things you don't need.*

THE TENSION DIAGRAM FOR COMPONENT COHESION

You may have already realized that the three cohesion principles tend to fight each other. The REP and CCP are *inclusive* principles: Both tend to make components larger. The CRP is an *exclusive* principle, driving components to be smaller. It is the tension between these principles that good architects seek to resolve.

Figure 13.1 is a tension diagram[2] that shows how the three principles of cohesion interact with each other. The edges of the diagram describe the *cost* of abandoning the principle on the opposite vertex.

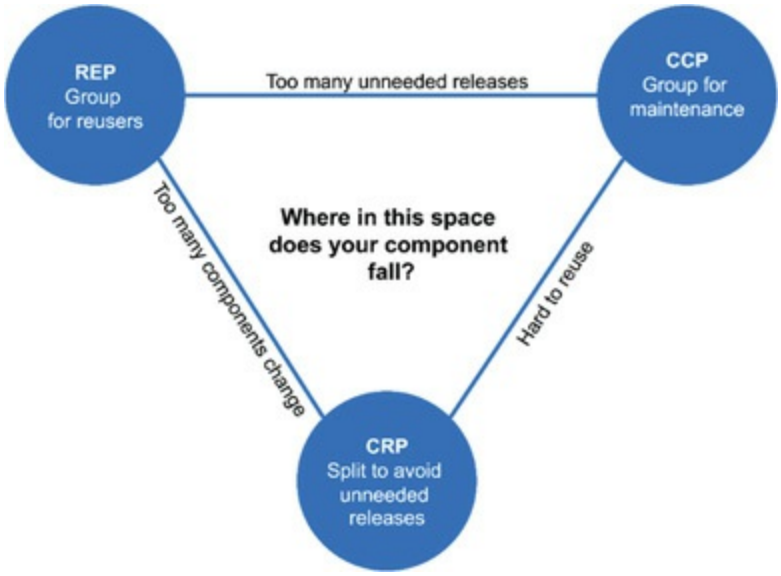

Figure 13.1 Cohesion principles tension diagram

An architect who focuses on just the REP and CRP will find that too many components are impacted when simple changes are made. In contrast, an architect who focuses too strongly on the CCP and REP will cause too many unneeded releases to be generated.

A good architect finds a position in that tension triangle that meets the *current* concerns of the development team, but is also aware that those concerns will change over time. For example, early in the development of a project, the CCP is much more important than the REP, because develop-ability is more important than reuse.

Generally, projects tend to start on the right hand side of the triangle, where the only sacrifice is reuse. As the project matures, and other projects begin to draw from it, the project will slide over to the left. This means that the component structure of a project can vary with time and maturity. It has more to do with the way that project is developed and used, than with what the project actually does.

CONCLUSION

In the past, our view of cohesion was much simpler than the REP, CCP, and CRP implied. We once thought that cohesion was simply the attribute that a module performs one, and only one, function. However, the three principles of component cohesion describe a much more complex variety of cohesion. In choosing the classes to group together into components, we must consider the opposing forces involved in reusability and develop-ability. Balancing these forces with the needs of the

application is nontrivial. Moreover, the balance is almost always dynamic. That is, the partitioning that is appropriate today might not be appropriate next year. As a consequence, the composition of the components will likely jitter and evolve with time as the focus of the project changes from develop-ability to reusability.

1. See the section on “The Kitty Problem” in Chapter 27, “Services: Great and Small.“
2. Thanks to Tim Ottinger for this idea.

14
COMPONENT COUPLING

The next three principles deal with the relationships between components. Here again we will run into the tension between develop-ability and logical design. The forces that impinge upon the architecture of a component structure are technical, political, and volatile.

THE ACYCLIC DEPENDENCIES PRINCIPLE

Allow no cycles in the component dependency graph.

Have you ever worked all day, gotten some stuff working, and then gone home, only to arrive the next morning to find that your stuff no longer works? Why doesn't it

work? Because somebody stayed later than you and changed something you depend on! I call this “the morning after syndrome.”

The “morning after syndrome” occurs in development environments where many developers are modifying the same source files. In relatively small projects with just a few developers, it isn’t too big a problem. But as the size of the project and the development team grow, the mornings after can get pretty nightmarish. It is not uncommon for weeks to go by without the team being able to build a stable version of the project. Instead, everyone keeps on changing and changing their code trying to make it work with the last changes that someone else made.

Over the last several decades, two solutions to this problem have evolved, both of which came from the telecommunications industry. The first is “the weekly build,” and the second is the Acyclic Dependencies Principle (ADP).

THE WEEKLY BUILD

The weekly build used to be common in medium-sized projects. It works like this: All the developers ignore each other for the first four days of the week. They all work on private copies of the code, and don’t worry about integrating their work on a collective basis. Then, on Friday, they integrate all their changes and build the system.

This approach has the wonderful advantage of allowing the developers to live in an isolated world for four days out of five. The disadvantage, of course, is the large integration penalty that is paid on Friday.

Unfortunately, as the project grows, it becomes less feasible to finish integrating the project on Friday. The integration burden grows until it starts to overflow into Saturday. A few such Saturdays are enough to convince the developers that integration should really begin on Thursday—and so the start of integration slowly creeps toward the middle of the week.

As the duty cycle of development versus integration decreases, the efficiency of the team decreases, too. Eventually this situation becomes so frustrating that the developers, or the project managers, declare that the schedule should be changed to a biweekly build. This suffices for a time, but the integration time continues to grow with project size.

Eventually, this scenario leads to a crisis. To maintain efficiency, the build schedule

has to be continually lengthened—but lengthening the build schedule increases project risks. Integration and testing become increasingly harder to do, and the team loses the benefit of rapid feedback.

ELIMINATING DEPENDENCY CYCLES

The solution to this problem is to partition the development environment into releasable components. The components become units of work that can be the responsibility of a single developer, or a team of developers. When developers get a component working, they release it for use by the other developers. They give it a release number and move it into a directory for other teams to use. They then continue to modify their component in their own private areas. Everyone else uses the released version.

As new releases of a component are made available, other teams can decide whether they will immediately adopt the new release. If they decide not to, they simply continue using the old release. Once they decide that they are ready, they begin to use the new release.

Thus no team is at the mercy of the others. Changes made to one component do not need to have an immediate affect on other teams. Each team can decide for itself when to adapt its own components to new releases of the components. Moreover, integration happens in small increments. There is no single point in time when all developers must come together and integrate everything they are doing.

This is a very simple and rational process, and it is widely used. To make it work successfully, however, you must *manage* the dependency structure of the components. *There can be no cycles*. If there are cycles in the dependency structure, then the “morning after syndrome” cannot be avoided.

Consider the component diagram in Figure 14.1. It shows a rather typical structure of components assembled into an application. The function of this application is unimportant for the purpose of this example. What *is* important is the dependency structure of the components. Notice that this structure is a *directed graph*. The components are the *nodes*, and the dependency relationships are the *directed edges*.

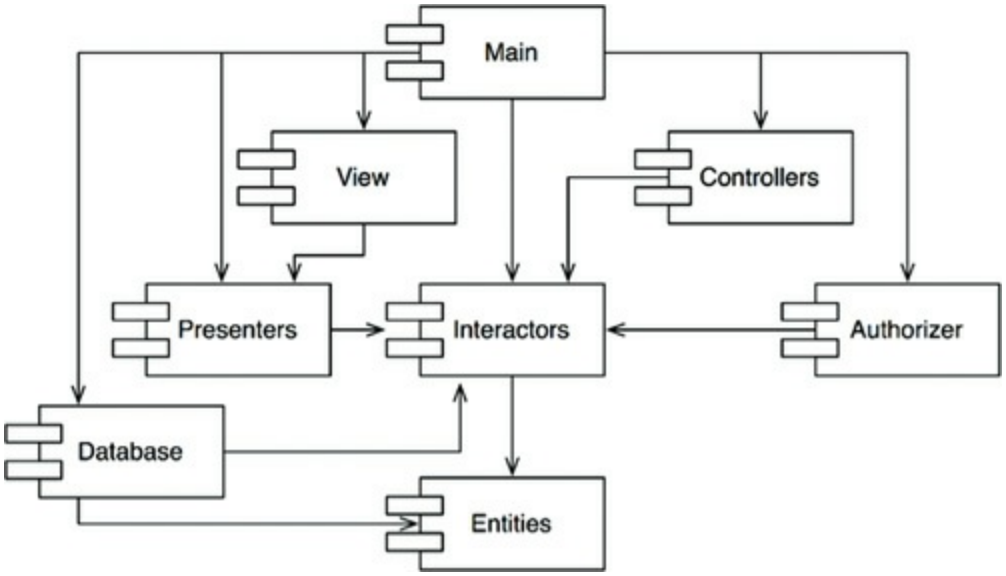

Figure 14.1 Typical component diagram

Notice one more thing: Regardless of which component you begin at, it is impossible to follow the dependency relationships and wind up back at that component. This structure has no cycles. It is a *directed acyclic graph* (DAG).

Now consider what happens when the team responsible for `Presenters` makes a new release of their component. It is easy to find out who is affected by this release; you just follow the dependency arrows backward. Thus `View` and `Main` will both be affected. The developers currently working on those components will have to decide when they should integrate their work with the new release of `Presenters`.

Notice also that when `Main` is released, it has utterly no effect on any of the other components in the system. They don't know about `Main`, and they don't care when it changes. This is nice. It means that the impact of releasing `Main` is relatively small.

When the developers working on the `Presenters` component would like to run a test of that component, they just need to build their version of `Presenters` with the versions of the `Interactors` and `Entities` components that they are currently using. None of the other components in the system need be involved. This is nice. It means that the developers working on `Presenters` have relatively little work to do to set up a test, and that they have relatively few variables to consider.

When it is time to release the whole system, the process proceeds from the bottom up. First the `Entities` component is compiled, tested, and released. Then the same is done for `Database` and `Interactors`. These components are followed by `Presenters`, `View`, `Controllers`, and then `Authorizer`. `Main` goes last. This process

is very clear and easy to deal with. We know how to build the system because we understand the dependencies between its parts.

THE EFFECT OF A CYCLE IN THE COMPONENT DEPENDENCY GRAPH

Suppose that a new requirement forces us to change one of the classes in `Entities` such that it makes use of a class in `Authorizer`. For example, let's say that the `User` class in `Entities` uses the `Permissions` class in `Authorizer`. This creates a dependency cycle, as shown in Figure 14.2.

This cycle creates some immediate problems. For example, the developers working on the `Database` component know that to release it, the component must be compatible with `Entities`. However, with the cycle in place, the `Database` component must now *also* be compatible with `Authorizer`. But `Authorizer` depends on `Interactors`. This makes `Database` much more difficult to release. `Entities`, `Authorizer`, and `Interactors` have, in effect, become one large component—which means that all of the developers working on any of those components will experience the dreaded "morning after syndrome." They will be stepping all over one another because they must all use exactly the same release of one another's components.

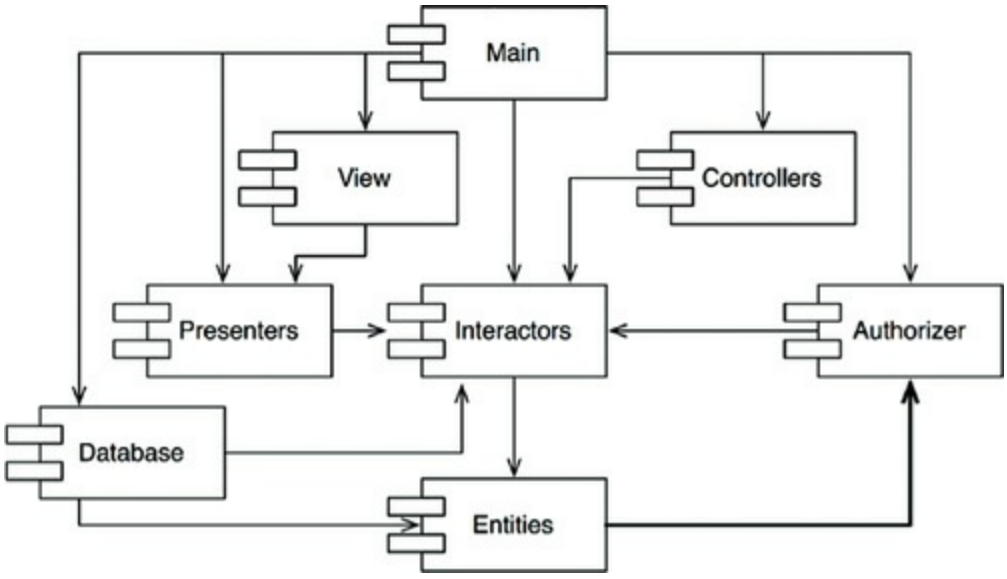

Figure 14.2 A dependency cycle

But this is just part of the trouble. Consider what happens when we want to test the `Entities` component. To our chagrin, we find that we must build and integrate with `Authorizer` and `Interactors`. This level of coupling between components is

troubling, if not intolerable.

You may have wondered why you have to include so many different libraries, and so much of everybody else's stuff, just to run a simple unit test of one of your classes. If you investigate the matter a bit, you will probably discover that there are cycles in the dependency graph. Such cycles make it very difficult to isolate components. Unit testing and releasing become very difficult and error prone. In addition, build issues grow geometrically with the number of modules.

Moreover, when there are cycles in the dependency graph, it can be very difficult to work out the order in which you must build the components. Indeed, there probably is no correct order. This can lead to some very nasty problems in languages like Java that read their declarations from compiled binary files.

BREAKING THE CYCLE

It is always possible to break a cycle of components and reinstate the dependency graph as a DAG. There are two primary mechanisms for doing so:

1. Apply the Dependency Inversion Principle (DIP). In the case in Figure 14.3, we could create an interface that has the methods that `User` needs. We could then put that interface into `Entities` and inherit it into `Authorizer`. This inverts the dependency between `Entities` and `Authorizer`, thereby breaking the cycle.

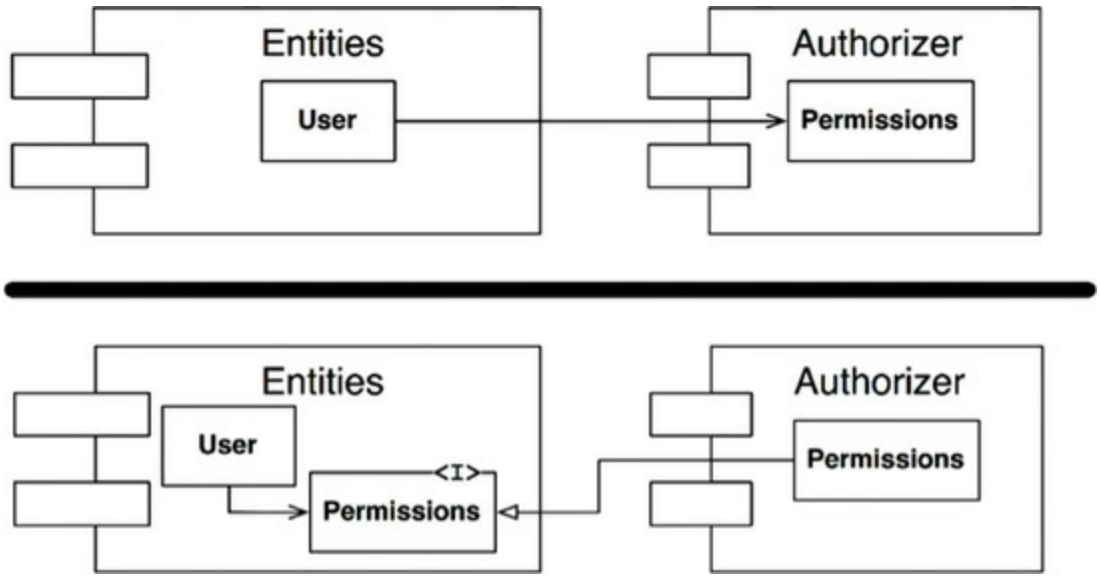

Figure 14.3 Inverting the dependency between `Entities` and `Authorizer`

2. Create a new component that both `Entities` and `Authorizer` depend on. Move

the class(es) that they both depend on into that new component (Figure 14.4).

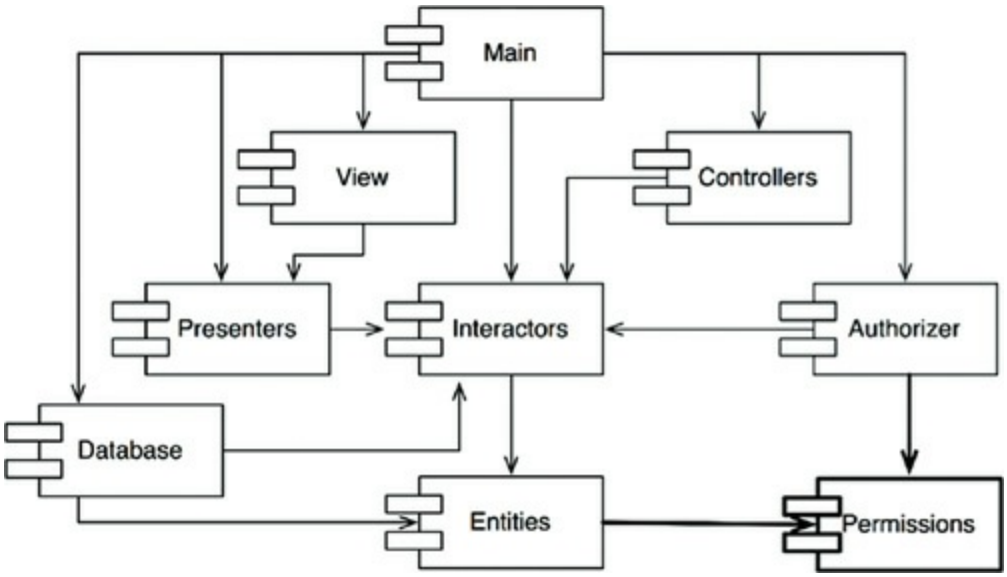

Figure 14.4 The new component that both `Entities` and `Authorizer` depend on

THE "JITTERS"

The second solution implies that the component structure is volatile in the presence of changing requirements. Indeed, as the application grows, the component dependency structure jitters and grows. Thus the dependency structure must always be monitored for cycles. When cycles occur, they must be broken somehow. Sometimes this will mean creating new components, making the dependency structure grow.

TOP-DOWN DESIGN

The issues we have discussed so far lead to an inescapable conclusion: The component structure cannot be designed from the top down. It is not one of the first things about the system that is designed, but rather evolves as the system grows and changes.

Some readers may find this point to be counterintuitive. We have come to expect that large-grained decompositions, like components, will also be high-level *functional* decompositions.

When we see a large-grained grouping such as a component dependency structure,

we believe that the components ought to somehow represent the functions of the system. Yet this does not seem to be an attribute of component dependency diagrams.

In fact, component dependency diagrams have very little do to with describing the function of the application. Instead, they are a map to the *buildability* and *maintainability* of the application. This is why they aren't designed at the beginning of the project. There is no software to build or maintain, so there is no need for a build and maintenance map. But as more and more modules accumulate in the early stages of implementation and design, there is a growing need to manage the dependencies so that the project can be developed without the "morning after syndrome." Moreover, we want to keep changes as localized as possible, so we start paying attention to the SRP and CCP and collocate classes that are likely to change together.

One of the overriding concerns with this dependency structure is the isolation of volatility. We don't want components that change frequently and for capricious reasons to affect components that otherwise ought to be stable. For example, we don't want cosmetic changes to the GUI to have an impact on our business rules. We don't want the addition or modification of reports to have an impact on our highest-level policies. Consequently, the component dependency graph is created and molded by architects to protect stable high-value components from volatile components.

As the application continues to grow, we start to become concerned about creating reusable elements. At this point, the CRP begins to influence the composition of the components. Finally, as cycles appear, the ADP is applied and the component dependency graph jitters and grows.

If we tried to design the component dependency structure before we designed any classes, we would likely fail rather badly. We would not know much about common closure, we would be unaware of any reusable elements, and we would almost certainly create components that produced dependency cycles. Thus the component dependency structure grows and evolves with the logical design of the system.

THE STABLE DEPENDENCIES PRINCIPLE

Depend in the direction of stability.

Designs cannot be completely static. Some volatility is necessary if the design is to

be maintained. By conforming to the Common Closure Principle (CCP), we create components that are sensitive to certain kinds of changes but immune to others. Some of these components are *designed* to be volatile. We *expect* them to change.

Any component that we expect to be volatile should not be depended on by a component that is difficult to change. Otherwise, the volatile component will also be difficult to change.

It is the perversity of software that a module that you have designed to be easy to change can be made difficult to change by someone else who simply hangs a dependency on it. Not a line of source code in your module need change, yet your module will suddenly become more challenging to change. By conforming to the Stable Dependencies Principle (SDP), we ensure that modules that are intended to be easy to change are not depended on by modules that are harder to change.

STABILITY

What is meant by "stability"? Stand a penny on its side. Is it stable in that position? You would likely say "no." However, unless disturbed, it will remain in that position for a very long time. Thus stability has nothing directly to do with frequency of change. The penny is not changing, but it is difficult to think of it as stable.

Webster's Dictionary says that something is stable if it is "not easily moved." Stability is related to the amount of work required to make a change. On the one hand, the standing penny is not stable because it requires very little work to topple it. On the other hand, a table is very stable because it takes a considerable amount of effort to turn it over.

How does this relate to software? Many factors may make a software component hard to change—for example, its size, complexity, and clarity, among other characteristics. We will ignore all those factors and focus on something different here. One sure way to make a software component difficult to change, is to make lots of other software components depend on it. A component with lots of incoming dependencies is very stable because it requires a great deal of work to reconcile any changes with all the dependent components.

The diagram in Figure 14.5 shows `X`, which is a stable component. Three components depend on `X`, so it has three good reasons not to change. We say that `X` is *responsible* to those three components. Conversely, `X` depends on nothing, so it has no external influence to make it change. We say it is *independent*.

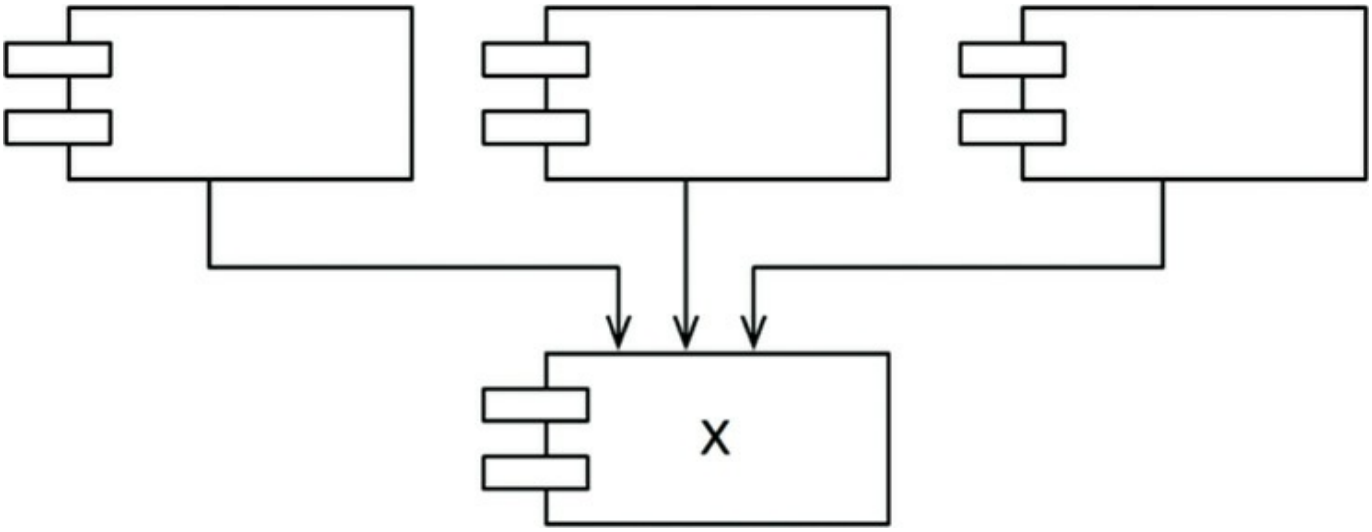

Figure 14.5 `X`: a stable component

Figure 14.6 shows `Y`, which is a very unstable component. No other components depend on `Y`, so we say that it is irresponsible. `Y` also has three components that it depends on, so changes may come from three external sources. We say that `Y` is dependent.

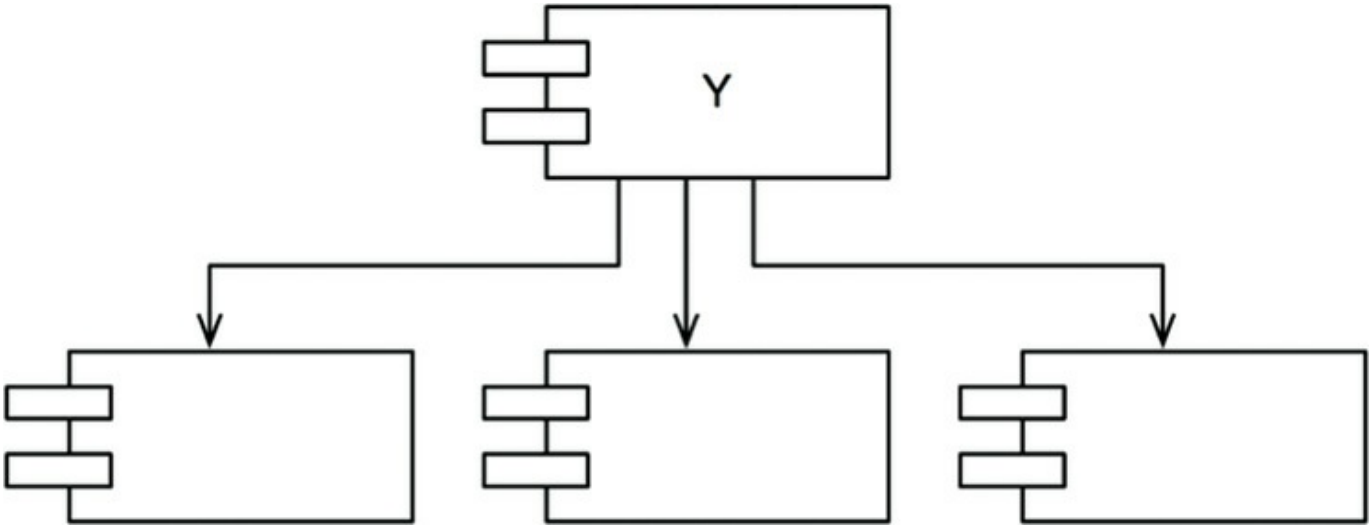

Figure 14.6 `Y`: a very unstable component

STABILITY METRICS

How can we measure the stability of a component? One way is to count the number of dependencies that enter and leave that component. These counts will allow us to calculate the *positional* stability of the component.

- *Fan-in*: Incoming dependencies. This metric identifies the number of classes outside this component that depend on classes within the component.
- *Fan-out*: Outgoing depenencies. This metric identifies the number of classes inside this component that depend on classes outside the component.
- *I*: Instability: $I = Fan\text{-}out\ ,\ (Fan\text{-}in + Fan\text{-}out)$. This metric has the range [0, 1]. $I = 0$ indicates a maximally stable component. $I = 1$ indicates a maximally unstable component.

The *Fan-in* and *Fan-out* metrics[1] are calculated by counting the number of *classes* outside the component in question that have dependencies with the classes inside the component in question. Consider the example in Figure 14.7.

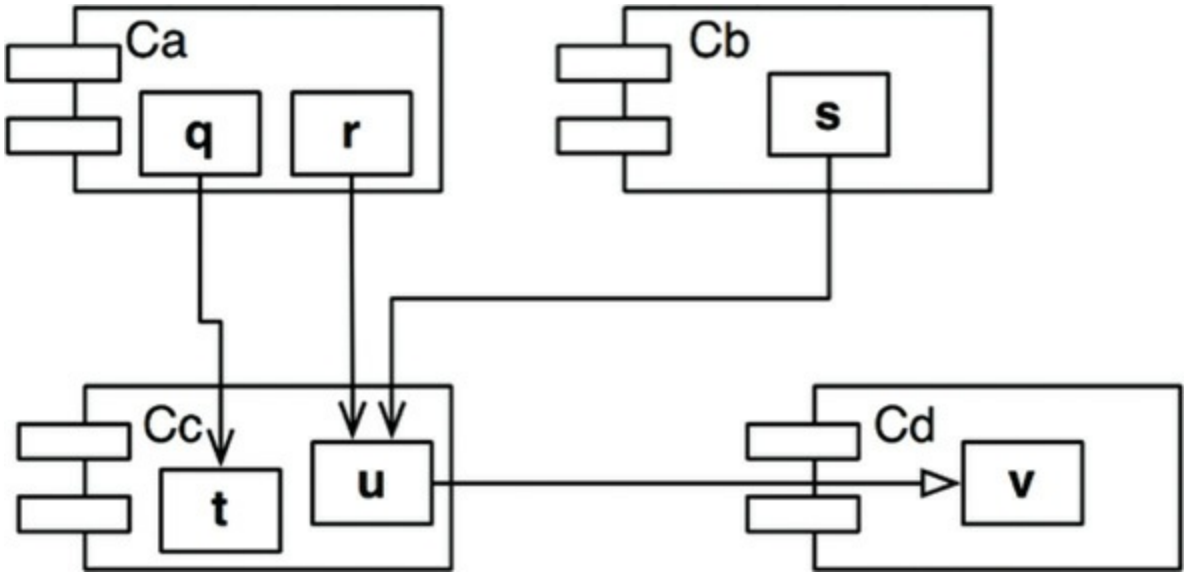

Figure 14.7 Our example

Let's say we want to calculate the stability of the component `Cc`. We find that there are three classes outside `Cc` that depend on classes in `Cc`. Thus, *Fan-in* = 3. Moreover, there is one class outside `Cc` that classes in `Cc` depend on. Thus, *Fan-out* = 1 and $I = 1/4$.

In C++, these dependencies are typically represented by `#include` statements. Indeed, the *I* metric is easiest to calculate when you have organized your source code such that there is one class in each source file. In Java, the *I* metric can be calculated by counting `import` statements and qualified names.

When the *I* metric is equal to 1, it means that no other component depends on this component (*Fan-in* = 0), and this component depends on other components (*Fan-out* > 0). This situation is as unstable as a component can get; it is irresponsible and